

SENIOR SWE INTERVIEW SERIES

Module 8: WebSockets, SSE, Polling & gRPC

Interview-focused networking, as asked at Google, Meta, Amazon, Microsoft, Uber, Netflix, Cloudflare, Stripe, LinkedIn & Cisco

Part of: Computer Networking for Senior Software Engineer Interviews

Built July 02, 2026

MODULE 8 — WebSockets, SSE, Polling, gRPC & HTTP Streaming

“How would you push updates to clients?” appears in chat-app, dashboard, notification, and collaborative-editor designs. The expected answer is a decision matrix across polling/SSE/WebSockets/gRPC — plus the ops reality of a million open connections.

Topic 8.1 — Short Polling

1. Why Interviewers Ask This

It's the baseline every alternative is measured against. Interviewers often *start* here (“simplest thing?”) and grade how precisely you cost it.

2. Core Concept

Client asks “anything new?” on a fixed interval (e.g., every 5s). Server answers immediately — usually “no.” Latency = up to one interval; cost = QPS × clients regardless of activity.

3. Internal Working

Each poll is a full HTTP request: over keep-alive connections the TCP/TLS cost amortizes, but you still pay headers (mitigated by h2/HPACK), auth verification, routing, handler execution, and a DB/queue check per poll per client. Stateless — any backend can answer; this operational simplicity is its entire value proposition.

4. Packet Flow Explanation

```
every 5s: GET /messages?since=cursor_182
        -> LB -> any backend -> check store -> 200 [] (empty, ~200B)
message actually arrives 2.4s after a poll => user sees it up to 5s late
scale math: 1M clients / 5s = 200,000 QPS of mostly-empty responses
```

5. ASCII Diagram

```
client: |--poll--|...5s...|--poll--|...5s...|--poll--|
server:   "no"           "no"           "yes: 1 msg"
event  -----^ (waits 4.9s to be seen)
latency: U(0, interval)  cost: clients/interval QPS, always-on
```

6. Real Production Example

Still correct for: slow-changing dashboards (poll 30–60s), mobile background sync (OS batches radio wakeups), cron-like status checks, third-party APIs offering nothing better. Many “real-time” admin panels are 10s short-polls and nobody notices.

7. Advantages

Trivially stateless/cacheable/load-balanced; works through every proxy/firewall ever made; failure recovery = the next poll (self-healing); backpressure-free.

8. Trade-offs

Latency floor = interval/2 average; wasted work scales with users not activity; battery/radio cost on mobile; thundering-herd if intervals synchronize (jitter!).

9. Common Mistakes

Dismissing it reflexively ("polling is bad") — interviewers respect cost-based reasoning; polling with no jitter (synchronized stampedes); not using conditional requests (ETag/If-None-Match → 304s make empty polls nearly free).

10. Performance Impact

Cost model to recite: $\text{QPS} = N/\text{interval}$; $\text{bytes} \approx \text{QPS} \times (\text{headers} + \text{empty body})$; with ETag/304 + h2 an empty poll can be <100 bytes and ~0 backend work if the check is a cache hit. Latency: avg interval/2, worst interval.

11. Common Interview Questions

1. When is short polling the *right* answer?
2. Cost 1M clients polling every 5s.
3. How do you cheapen empty polls? (304s, cursors, edge-cached "latest version" checks.)

12. Follow-up Questions

"How do you prevent synchronized polling?" → jitter $\pm 20\%$, per-client offsets. "Poll frequency vs freshness SLO?" → interval $\approx$ freshness budget; below ~2s intervals, switch technology.

13. Debugging Scenarios

Traffic sawtooth every N sec → synchronized clients (deploys reset timers!); DB load scales with logins not messages → polls hitting the primary; add cache/materialized cursor.

14. Best Practices

Jitter always; ETag/cursor-based cheap checks; adaptive intervals (back off when idle); treat >0.5 Hz polling as a smell to upgrade to SSE/WS.

15. Practice Questions

1. Dashboard freshness SLO 30s, 50k viewers: design the poll (interval 20s+jitter, 304-friendly endpoint, edge cache 15s → origin ~sub-100 QPS).
2. Why might 1s short polling beat WebSockets for a 99.9%-idle IoT fleet behind hostile NATs? (No conn state to keep alive; NAT timeouts irrelevant; radio batching.)

Topic 8.2 — Long Polling

1. Why Interviewers Ask This

The historical bridge to real-time (Comet-era) and still the fallback under WebSocket-hostile networks. Tests understanding of held connections and their infrastructure costs.

2. Core Concept

Client sends a request; server **holds it open** (25–60s) until data exists or timeout, then responds; client immediately re-requests. Latency $\approx$ instant (data pushed on the held request); idle cost = held connections instead of QPS.

3. Internal Working

Server-side: the handler parks (async/event-loop — thread-per-request stacks die here: 100k held requests = 100k threads unless async) subscribed to a pub/sub topic; on event $\rightarrow$ respond $\rightarrow$ client loops. Every intermediary's timeout must exceed the hold time (LB idle, proxy read, gateway) — the #1 deployment failure. Missed-event race: events arriving *between* polls must be covered by a cursor/sequence in the next request.

4. Packet Flow Explanation

```
GET /events?cursor=182 -----> server parks request (subscribes)
...silence 0-30s (heartbeat/timeout at 30s -> 204 -> re-poll)...
event #183 published -> server responds 200 [183] immediately
client processes, loops: GET /events?cursor=183 (gap-free via cursor)
p50 event latency ~ one-way network time; cost: 1 held conn/client
```

5. ASCII Diagram

```
client: |--req-----held(28s)-----200[data]|--req---held...
server:      (parked, subscribed) ^event fires
vs short poll: latency ~0, empty responses ~0
vs websocket: still request/response; new request per message batch;
               full HTTP semantics preserved (auth per request!)
```

6. Real Production Example

Basis of the Comet era (Gmail chat), Facebook chat pre-WS, and still today: Kafka consumer `poll(timeout)` semantics, AWS SQS long polling (`WaitTimeSeconds=20` — cuts empty-receive billing), Consul/etcd blocking queries for config watches. Long polling never died; it moved to infrastructure APIs.

7. Advantages

Near-real-time latency with plain HTTP (every proxy/firewall/LB/auth middleware just works); per-request auth/routing (no long-lived session semantics); graceful under horizontal scale (each poll can land anywhere if the pub/sub layer is shared).

8. Trade-offs

Held-connection capacity planning (fd, memory per parked request); timeout choreography across every hop; message batching awkward (respond per event or micro-batch?); reconnect gap races require cursors; $\sim 2\times$ message overhead vs WS for chatty streams.

9. Common Mistakes

Hold time > some intermediary's idle timeout (random 504s/resets); thread-per-request servers (thread starvation at trivial user counts); no cursor → events lost in the re-poll gap; treating it as obsolete (SQS!).

10. Performance Impact

Event latency $\approx$ network one-way (vs interval/2); server cost shifts from CPU (poll QPS) to memory/fds (parked conns $\sim$ 10KB each async). 100k idle clients: short-poll@5s = 20k QPS vs long-poll = 100k parked conns + $\sim$ 3.3k re-poll QPS — know how to compare these regimes.

11. Common Interview Questions

1. Short vs long polling — cost regimes.
2. What breaks long polling in real deployments? (Timeout chain.)
3. How do you guarantee no missed events across re-polls? (Cursors/seq numbers, server buffers.)

12. Follow-up Questions

“Why does SQS long polling save money?” → fewer empty receives (billed per request). “How does the server scale parked requests?” → async I/O, pub/sub fan-out, connection-count-based autoscaling (not CPU!).

13. Debugging Scenarios

Clients report events in bursts of 30s → responses only flushing at timeout (missing event → response wiring or response buffering at a proxy — disable proxy buffering!); 502 storms at exactly 60s → ALB idle timeout < hold time.

14. Best Practices

Hold $\approx$ 20–30s (safely under common 60s timeouts) + heartbeat/204; cursors mandatory; async runtime; jittered immediate re-poll; disable response buffering on the path.

15. Practice Questions

1. Design long polling for 500k mobile clients: hold time vs mobile NAT timeouts ($\sim$ 30s UDP but TCP NAT $\sim$ 15min — hold 25s is fine), fleet sizing by fd/memory, cursor recovery, and the LB timeout config.
2. SQS charges per request. Compute monthly cost delta: 100 queues polled by 50 consumers, short poll 1s vs long poll 20s at $\sim$ 1 msg/min/queue.

Topic 8.3 — SSE (Server-Sent Events)

1. Why Interviewers Ask This

SSE is the deliberately-underrated option interviewers use to test breadth: one-directional push with plain HTTP — and since ChatGPT, *the* streaming mechanism everyone has seen (token streaming is SSE-style).

2. Core Concept

One long-lived HTTP response (`Content-Type: text/event-stream`) that the server writes forever: `data:` lines separated by blank lines; optional `id:` (enables resume via `Last-Event-ID` header) and

event: types. Browser-native `EventSource` API with **built-in auto-reconnect + resume**. Server → client only; client → server goes over normal separate requests.

3. Internal Working

It's just a never-ending chunked/streamed HTTP response — which is why it traverses proxies (with buffering off) and works over h2/h3 (each SSE stream = one h2 stream — the 6-connection h1 limit vanishes under h2; know this evolution). Reconnect: browser re-GETs with `Last-Event-ID: 47` → server replays from its per-topic ring buffer → gap-free delivery cheaply. Text-only, UTF-8 (binary needs base64 or use WS).

4. Packet Flow Explanation

```
GET /stream Accept:text/event-stream
<- 200, Content-Type: text/event-stream (response never completes)
<- data:{"price":101.2}\n\n ... <- id:48\ndata:{...}\n\n
[network blip] EventSource auto-reconnects:
GET /stream Last-Event-ID:48 -> server replays 49..now, resumes live
heartbeat comment ":ka\n\n" every 15-30s keeps proxies/NAT from
declaring the response dead.
```

5. ASCII Diagram

```
client --GET--> [proxy: buffering OFF] --> server
<===== one endless response =====>
data: ... data: ... :heartbeat data: ...
direction: server->client ONLY (client speaks via normal POSTs)
free gifts: auto-reconnect, Last-Event-ID resume, HTTP auth/infra reuse
```

6. Real Production Example

LLM token streaming (OpenAI/Anthropic APIs stream completions as SSE), live scores/tickers, CI log streaming, feature-flag/config push (many vendors), notification feeds. Twitter's early streaming APIs; GraphQL subscriptions over SSE gaining ground precisely for infra simplicity.

7. Advantages

Plain HTTP: auth, LBs, CDNs (some can even fan out SSE at edge), h2 multiplexing, no protocol upgrade dance; built-in reconnect+resume (WS gives you neither!); simplest server code of all push options.

8. Trade-offs

One-way only; text-only; requires buffering disabled on every hop + heartbeats; h1 legacy limit (6 conns/host) if h2 unavailable; no built-in backpressure signal from client (slow client → TCP backpressure → server must drop/close).

9. Common Mistakes

Choosing WS “for real-time” when data flow is one-directional (dashboards, feeds, notifications — SSE is operationally cheaper: the mistake interviewers *want* you to call out); forgetting proxy buffering (`X-Accel-Buffering: no` / `proxy_buffering off`) — symptoms: events arrive in giant delayed clumps; no heartbeat → 60s idle kills.

10. Performance Impact

Per-message overhead $\approx$ bytes + `data:` framing (no per-message HTTP cost); connection cost $\approx$ WS (~one socket + ~10KB); resume avoids re-fetch storms after blips. Latency = one-way network (same as WS for server $\rightarrow$ client).

11. Common Interview Questions

1. SSE vs WebSockets — when is SSE strictly better? 2. How does reconnection/resume work? 3. How does SSE behave over h1 vs h2?

12. Follow-up Questions

“How do you scale SSE fan-out to 1M clients?” $\rightarrow$ same as WS: pub/sub backbone (Redis/Kafka) $\rightarrow$ edge fan-out tier holding sockets; per-topic ring buffers for resume. “Client $\rightarrow$ server channel?” $\rightarrow$ ordinary POSTs — and that’s fine (chat apps have asymmetric volume: read $\gg$ write).

13. Debugging Scenarios

Events arrive batched every ~4KB $\rightarrow$ proxy/gzip buffering (disable compression or flush-per-event); connections die at 60s idle $\rightarrow$ missing heartbeat vs proxy read timeout; works locally, dead behind corporate proxy $\rightarrow$ ancient proxy buffering entire response: provide long-poll fallback.

14. Best Practices

Heartbeat 15–30s; `id:` on every event + replay buffer (size = max tolerated disconnect $\times$ event rate); disable buffering/compression on the route; cap per-connection queue and drop-slowest policy; h2 for browsers.

15. Practice Questions

1. Design live-order-status streaming for a food-delivery app (mobile+web). Why SSE over WS here? (One-way status flow; POSTs for actions; reconnect/resume free; CDN-friendly.)
2. Your SSE stream must survive LB failovers with zero missed events. Design the id/replay contract and size the buffer for a 5-min outage at 10 events/s.

Topic 8.4 — WebSockets

1. Why Interviewers Ask This

The default “real-time” answer — so interviewers drill the parts people skip: the upgrade handshake, proxy traversal, keep-alive/backpressure, and the *stateful-fleet* operational burden (draining, reconnect storms).

2. Core Concept

A single TCP connection upgraded from HTTP into a **full-duplex, message-framed, persistent** channel (RFC 6455). Either side sends anytime. Binary+text frames. No request/response semantics, no auto-reconnect, no resume — *you* build the protocol on top (this sentence wins interviews).

3. Internal Working

- Handshake: `GET` + `Upgrade: websocket` + `Sec-WebSocket-Key` → `101 Switching Protocols` + hashed key echo (proves a WS-aware server, prevents cache/proxy confusion). After 101, HTTP is *gone* — raw WS frames on the socket.
- Frames: FIN/opcode (text/binary/ping/pong/close), length, client → server frames masked (XOR — defeats broken transparent-proxy cache poisoning; a classic “why masking?” follow-up).
- h2/h3: RFC 8441 tunnels WS over an h2 stream (`:protocol=websocket` extended CONNECT); support is uneven — most real deployments are still WS-over-h1.1 per connection.
- Ping/pong frames = protocol-level keep-alive + liveness (use them; TCP KA is invisible to L7).

4. Packet Flow Explanation

```
GET /chat HTTP/1.1
Upgrade: websocket | Connection: Upgrade | Sec-WebSocket-Key: q4x...
<- HTTP/1.1 101 Switching Protocols | Sec-WebSocket-Accept: hash
[socket is now WS]
c->s frame: {type:"msg", room:7, text:"hi"}    (masked)
s->c frame: {type:"msg", from:"bob", ...}      (any time - full duplex)
every 30s: ping -> pong (dead-peer detection + NAT/proxy freshness)
close: close frame code 1000 (or 1001 going-away on deploys) -> TCP FIN
```

5. ASCII Diagram

```
HTTP hop-by-hop world:      WS world (after 101):
[client]->[LB]->[proxy]->[srv]  [client]<====duplex frames====>[srv]
each hop must FORWARD the upgrade  every hop must now be a dumb pipe
(ALB: yes; nginx: explicit          with LONG idle timeouts
 Upgrade/Connection headers)
state: connection lives on ONE server -> sticky by nature ->
draining, reconnect storms, presence state = your problems now
```

6. Real Production Example

Slack/Discord (chat+presence: Discord runs millions of WS on Elixir/Erlang gateways), Figma (collaborative editing, binary frames + CRDT/OT ops), trading UIs, multiplayer lobbies, Kubernetes `kubectrl exec` (WS tunnels), GraphQL subscriptions. Discord's gateway design (identify → resume tokens → jittered reconnect) is the public case study worth citing.

7. Advantages

Lowest latency both directions; binary framing; one connection for unlimited logical channels (multiplex in your protocol); ideal for high-frequency bidirectional state (cursors, gameplay, collaborative editing).

8. Trade-offs

You own: reconnect w/ backoff+jitter, resume/sync-on-reconnect, heartbeats, auth-refresh mid-connection (token expires — then what? — great follow-up), multiplexing, backpressure (`bufferedAmount` /drop policies). Fleet is stateful: draining deploys, connection-count autoscaling, reconnect storms after LB/network blips. Some proxies still break WS → need SSE/long-poll fallback (socket.io's entire reason).

9. Common Mistakes

No jittered backoff → your own clients DDoS you after every blip (the classic reconnect storm); no server → client heartbeat → half-open zombie sockets pile up (NAT died silently — Module 2.9); unbounded

per-connection send queues → OOM from slow clients (drop or disconnect the slow — never buffer infinitely); auth only at handshake with no re-auth story.

10. Performance Impact

Per-conn memory: ~tens of KB (kernel buffers + app state) → 1M conns ≈ tens of GB fleet-wide (fine, but *planned*); frames ~2–14B overhead vs ~100s of B for HTTP requests → 10–100× cheaper per message for chatty flows; deploys: draining 1M conns with jittered **1001 going-away** over minutes vs instant kill = reconnect-storm math interviewers love.

11. Common Interview Questions

1. Walk the upgrade handshake; why 101, why the key hash, why masking?
2. Design chat for 10M concurrent: gateway tier (WS) + pub/sub backbone + presence + resume protocol.
3. What do you re-implement on top of WS that HTTP gave free? (Auth-per-request, routing, caching, reconnect, request-IDs/correlation.)

12. Follow-up Questions

“How do WS servers deploy without dropping the world?” → connection draining: stop accepting, send going-away with jitter, let clients reconnect to new fleet over N minutes. “How does the LB handle WS?” → it’s just a long-lived conn (L4 easy; L7 must support upgrade + long idle timeouts); balancing is per-connection → watch skew. “Backpressure?” → monitor socket send queue; policy per stream type: drop-oldest (telemetry) vs disconnect (state sync must not silently gap).

13. Debugging Scenarios

Connections die at exactly 60s idle → some hop’s idle timeout vs missing ping interval; upgrade fails through one proxy (400/502 on handshake) → **Upgrade / Connection** headers not forwarded; slow ramp of memory on gateways → zombie conns (no heartbeat) or unbounded queues — inspect per-conn buffered bytes.

14. Best Practices

Heartbeat both directions (~30s) < min idle timeout on path; resume protocol with session tokens + server-side event buffer; jittered exponential reconnect with server-controlled backoff hints; per-conn send-queue caps with explicit drop policy; fallback transport; connection-count (not CPU) autoscaling signals; drain playbooks.

15. Practice Questions

1. Size a WS gateway fleet for 5M conns, 2 msg/s avg, 8KB/conn state: memory, fd limits, per-box conn cap (~100–200k realistic), and the reconnect-storm plan when a box dies (5M/50 boxes = 100k clients reconnecting — jitter over how long?).
2. Design mid-session re-auth: JWT expires hourly on a 6-hour WS session. (In-band auth frame with refreshed token; server tracks expiry per conn; grace + close 4401 on failure.)

Topic 8.5 — gRPC

1. Why Interviewers Ask This

The dominant internal-RPC choice (Google-origin, CNCF standard). Interviews probe: what it actually is on the wire (h2 mapping), the four streaming modes, deadline/cancellation semantics, and the LB problem (Module 6 crossover).

2. Core Concept

gRPC = **Protobuf-defined contracts** (codegen for clients/servers in every language) carried as **HTTP/2 streams**: one RPC = one h2 stream; binary protobuf payloads; trailers carry status. Four shapes: unary, server-streaming, client-streaming, bidirectional streaming. First-class: deadlines (propagated!), cancellation, metadata, interceptors.

3. Internal Working

Wire mapping (know this cold): request = **HEADERS** (`:path=/pkg.Service/Method` , `content-type: application/grpc` , `grpc-timeout`) + **DATA** frames (5-byte prefix: compressed-flag + length, then protobuf) + half-close; response = **HEADERS** + **DATA** + **trailers** (`grpc-status` , `grpc-message`) — trailers are why plain browsers can't speak native gRPC (fetch API can't read trailers) → **gRPC-Web** (proxy translates) or **Connect/JSON transcoding**. **Deadline propagation**: client sets 500ms → every downstream hop receives remaining budget and aborts work when exceeded — the single best microservice-latency hygiene mechanism; cancellation cascades via **RST_STREAM**.

4. Packet Flow Explanation

```
one TCP+TLS conn (h2), three concurrent RPCs = streams 1,3,5:
s1: HEADERS(/users.Users/Get, grpc-timeout:400m) DATA(proto) END_STREAM
s3: HEADERS(/feed.Feed/Stream) ... server DATA DATA DATA (server-streaming)
s5: bidi chat: both sides interleave DATA frames independently
s1 response: HEADERS(200) DATA(proto) TRAILERS(grpc-status:0)
deadline exceeded at any hop -> RST_STREAM + grpc-status:4 upstream,
work stops downstream. PING frames keep the conn validated (keepalive).
```

5. ASCII Diagram

```
.proto contract -> codegen -> [client stub]      [server impl]
                               |                  |
                               h2 stream(s) over ONE warm conn
unary: req->resp | server-stream: req->resp* | client-stream: req*->resp
bidi: req* <-> resp* (independent)
status in TRAILERS -> browser needs gRPC-Web proxy
LB TRAP: 1 conn = 1 backend at L4 -> use L7/mesh/client-side LB
         + MAX_CONNECTION_AGE to rebalance
```

6. Real Production Example

Google internal (Stubby → gRPC), Netflix, Uber, Square, Dropbox for service-to-service; Kubernetes (etcd, CRI, CSI are gRPC); Envoy's xDS config protocol = gRPC streaming. Uber and Spotify publicly documented REST → gRPC migrations citing type-safety + p99 wins.

7. Advantages

~5–10× smaller payloads + faster serialization than JSON; contracts with codegen kill a whole bug class (and enable safe evolution via field numbers); streaming built-in; deadlines/cancellation as first-class citizens; interceptors standardize auth/retries/telemetry; one warm h2 conn (no per-request handshakes).

8. Trade-offs

Browser/humans need adapters (gRPC-Web, transcoding, grpcurl instead of curl); binary payloads = harder debugging on the wire; the L7-LB requirement; protobuf schema discipline required (field-number hygiene, never reuse numbers); h2 HoL blocking under loss persists (until gRPC-over-h3 matures).

9. Common Mistakes

gRPC behind L4 LB → one hot backend (say MAX_CONNECTION_AGE + L7/client-side LB); no deadlines set (defaults = infinite → thread/queue pileups during incidents — deadline-less gRPC is an outage pattern); treating streams as message-bus replacements (streams die with conns; use Kafka for durability); exposing raw gRPC publicly without gateway consideration.

10. Performance Impact

Benchmarks to quote (order-of-magnitude): protobuf encode/decode ~5–10× faster than JSON, payloads ~3–10× smaller; warm-conn unary RPC intra-DC p50 <1ms; streaming avoids per-message HTTP overhead entirely. Payload savings compound at fan-out (one request → 50 internal RPCs).

11. Common Interview Questions

1. gRPC vs REST — when each? (Internal/polyglot/streaming/performance → gRPC; public/browser/cacheable/simple → REST.)
2. The four streaming modes with a real use case each. (Unary=CRUD; server-stream=feeds/watches; client-stream=uploads/telemetry; bidi=chat/sync.)
3. Explain deadline propagation and why it beats per-hop timeouts.

12. Follow-up Questions

“Why HTTP/2 as substrate instead of custom TCP?” → multiplexing free, proxies/LBs speak it, TLS/ALPN infra reuse, streaming semantics map to streams. “How do retries work safely?” → only on idempotent methods / with retry policies + hedging + server-set `grpc-retry-pushback`; budget-capped. “Schema evolution rules?” → add fields with new numbers, never reuse/renumber, `reserved` deleted ones, no changing types in place.

13. Debugging Scenarios

p99 spikes with “DEADLINE_EXCEEDED” cascades → find the slow leaf via propagated deadline metrics, not the symptom services; `UNAVAILABLE` bursts on deploys → server not sending GOAWAY / clients not retrying on drain — enable graceful stop; one backend hot → connection imbalance (L4) — check conns-per-backend; use grpcurl + reflection to reproduce.

14. Best Practices

Deadlines on every call (derived from SLO budget), propagate always; MAX_CONNECTION_AGE (+grace) for rebalancing; interceptors for auth/tracing/retries in one place; health-check protocol (grpc.health.v1) wired to LB; keepalive PINGs tuned under proxy idle timeouts; version APIs via packages, evolve via field discipline.

15. Practice Questions

1. Migrate a REST monolith → gRPC microservices incrementally: transcoding gateway (REST ↔ gRPC), contract-first extraction order, deadline budget design (edge 800ms → leaf 100ms), rollout metrics.
2. Bidi-streaming sync service: define reconnect semantics (resume tokens in metadata), flow control (h2 windows + app-level acks), and why you still need a durable log behind it.

Topic 8.6 — HTTP Streaming (& the Decision Matrix)

1. Why Interviewers Ask This

Interviewers end this area with “so which one do you pick?” — they want a crisp decision procedure plus knowledge of plain HTTP streaming (chunked responses), which most candidates forget exists.

2. Core Concept

HTTP streaming = one response delivered progressively (h1 **Transfer-Encoding: chunked** / h2-h3 DATA frames): the server flushes bytes as produced — LLM tokens, big exports, video segments, progressive JSON (**application/x-ndjson**). SSE is a *formalized* HTTP stream; gRPC server-streaming is the same idea over h2 frames. Uploads stream the other way (client chunked request).

3. Internal Working

h1: each chunk = **<hex len>\r\n<bytes>\r\n**, terminated by **0\r\n\r\n** — receiver processes incrementally; h2/h3: DATA frames per stream, flow-control windows pace the producer (slow reader → window exhausts → server write blocks: built-in backpressure — nicer than SSE's TCP-only backpressure). Proxy buffering is again the arch-enemy: any hop buffering the response destroys the streaming property silently.

4. Packet Flow Explanation

```
POST /v1/completions          (LLM streaming, ndjson variant)
<- 200, Transfer-Encoding: chunked
<- {"token":"The"}\n  <- {"token":" cat"}\n  ... (flush per token)
client renders tokens as they arrive; TTFB ≈ first-token time (300ms)
instead of full-completion time (12s). Same infra as any HTTP request:
auth, LB, retries(on connect), CDN pass-through with buffering off.
```

5. ASCII Diagram — THE DECISION MATRIX (memorize)

```
need server->client push?
no  -> plain req/resp (+ cache)
yes -> how fresh?  minutes: SHORT POLL (+ETag)
                   seconds: LONG POLL (fallback tier)
                   instant, one-way: SSE (feeds, dashboards,
                                           notifications, LLM tokens)
                   instant, two-way, high-freq: WEBSOCKET
                                           (chat, games, collab editing)
service-to-service RPC/streams: gRPC (contracts, deadlines, h2)
one big/slow response: HTTP STREAMING (exports, tokens, progress)
cross-cutting: hostile networks -> keep an HTTP fallback (SSE/long-poll);
durability -> none of these are queues; pair with Kafka/etc.
```

6. Real Production Example

LLM APIs (chunked/SSE token streams — the defining 2023+ example); Twitter firehose (streaming HTTP of ndjson); S3/GCS large object downloads (range + streaming); Docker pull progress; Kubernetes `watch=true` (chunked JSON stream of resource changes — long-running HTTP streaming powering all controllers!).

7. Advantages

Zero new protocol: auth/observability/LB unchanged; TTFB decoupled from total time; natural for producer-paced data; h2 gives per-stream backpressure; trivially consumable (curl works).

8. Trade-offs

One-directional per response; connection failure mid-stream needs app-level resume (Range headers for bytes, cursors for events); intermediary buffering/timeouts; no message semantics on h1 (you invent framing: ndjson/length-prefix).

9. Common Mistakes

Building WS infrastructure for what is a single progressive response (exports, LLM output — streaming HTTP is simpler); forgetting resume for large downloads (Range/If-Range); Content-Length + streaming confusion (you don't know length → chunked; setting both breaks clients); proxy buffering yet again.

10. Performance Impact

Perceived latency: TTFB drops from total-time to first-chunk-time (LLM: 12s → 300ms user-perceived — the most vivid modern example); memory: $O(\text{chunk})$ instead of $O(\text{response})$ on both ends; long streams occupy conn slots — cap durations, resume via cursor.

11. Common Interview Questions

1. Compare all six mechanisms on: latency, direction, statefulness, infra-compat, resume. (Build the matrix live.)
2. How does chunked encoding work and when do you still need it under h2? (h2 frames replace it, but semantic remains: unknown-length progressive body.)
3. Design LLM response streaming end-to-end including proxy config and client parsing.

12. Follow-up Questions

“Backpressure in each?” → polling: none needed; SSE: TCP-only; WS: app-managed queues; gRPC/h2: stream windows (best). “Resume in each?” → poll: cursor param; SSE: Last-Event-ID; WS: your protocol; HTTP download: Range; gRPC: your resume token.

13. Debugging Scenarios

Stream “works but arrives all at once” → buffering (proxy, gzip, app framework auto-buffering — check every layer); mid-stream truncation at 100s → LB response timeout; client memory blowup → reading entire stream before processing (use incremental parser).

14. Best Practices

Flush explicitly per logical unit; heartbeat long streams; disable buffering/compression on streaming routes (or use per-chunk flush-friendly compression); pair every stream with a resume mechanism; enforce max stream duration + reconnect (rolling restarts need it).

15. Practice Questions

1. Notification system for web+mobile+3rd-party-webhooks: pick a mechanism per consumer type and justify (web: SSE; mobile: platform push (FCM/APNs — not your socket! battery/OS reality); partners: webhooks+retries; internal: Kafka).
2. 5GB report download must survive network changes: design with Range/If-Range + ETag pinning + parallel chunk fetch.

MODULE 8 — One-Page Cheat Sheet

DECISION	minutes-fresh: short poll(+jitter,+304) seconds: long poll one-way instant: SSE two-way high-freq: WebSocket s2s RPC/streams: gRPC one progressive response: HTTP stream mobile background: platform push (FCM/APNs), not sockets
SHORT POLL	QPS=N/interval; latency~interval/2; stateless; jitter!
LONG POLL	park 20-30s < ALL hop timeouts; cursors mandatory; async srv
SSE	text/event-stream; EventSource auto-reconnect + Last-Event-ID replay; heartbeat 15-30s; buffering OFF; one-way; h2 fixes 6-conn limit
WEBSOCKET	Upgrade->101; masked client frames; ping/pong; YOU build: reconnect(jitter!), resume, re-auth, backpressure(queue caps), drain(1001+jitter). stateful fleet; conn-count autoscaling
gRPC	proto contracts; 1 RPC = 1 h2 stream; status in TRAILERS (=> gRPC-Web for browsers); 4 modes; DEADLINES PROPAGATE; L4 LB trap -> L7/client LB + MAX_CONNECTION_AGE
HTTP STREAM	chunked/h2 DATA; flush per unit; TTFB=first chunk; Range resume; k8s watch, LLM tokens
UNIVERSAL	every hop's idle/read timeout > heartbeat interval;
GOTCHAS	proxy buffering kills streams; reconnect storms need jitter; none of these are durable queues — pair with a log (Kafka)
NUMBERS	WS conn ~10s KB => ~100-200k conns/box frame 2-14B vs HTTP ~500B proto ~5-10x vs JSON heartbeats ~30s

MODULE 8 — Top Interview Questions

1. Build the six-way comparison matrix from memory: direction, latency, state, resume, infra-compat, cost regime.
2. Design chat at 10M concurrent (gateway tier, pub/sub backbone, presence, resume, drain, reconnect storms).
3. WebSocket upgrade handshake — every header and why masking exists.
4. Why choose SSE over WS for a live dashboard? (One-way, auto-resume, plain HTTP ops.)
5. gRPC on the wire: streams, trailers, deadline propagation; why browsers can't and what gRPC-Web does.
6. The gRPC/L4 balancing problem and its three fixes.
7. What must every long-lived-connection design include? (Heartbeats, jittered reconnect, resume protocol, queue caps, drain plan, fallback transport.)
8. Why is mobile push (FCM/APNs) not just “keep a WebSocket open”? (OS kills sockets; battery; single multiplexed OS channel.)

MODULE 8 — Common Mistakes

- WS-by-default for one-way flows; polling dismissed without cost math.
- Missing heartbeats / timeout-chain audits; proxy buffering unexamined.
- Reconnects without jitter (self-DDoS); no resume protocol (silent gaps).
- Unbounded send queues to slow clients (OOM); auth solved only at handshake.
- gRPC without deadlines; behind L4; streams treated as durable queues.

MODULE 8 — Mock Interview (15 min)

Q1. “Design the realtime layer for a collaborative document editor.” *Strong answer:* WS (bidi, high-frequency ops); OT/CRDT ops as binary frames; per-doc room routing via consistent hash on docID to gateway shard (legit stickiness — Module 6.4); resume: op-log with vector/sequence per doc, replay on reconnect; presence via ephemeral pub/sub; drain + jittered reconnect; SSE/long-poll fallback; offline queue merges via CRDT properties.

Q2. “Your WS fleet dropped 2M connections during a 10s LB blip; the reconnect wave took the site down. Fix the system.” *Strong answer:* jittered exponential backoff with server-driven hints (retry-after in close frame), reconnect admission control (token-bucket at edge, shed excess to long-poll), resume tokens to skip re-auth+full-sync cost, connection-count-aware autoscaling with pre-warmed headroom, and staged LB maintenance (drain, don’t blip).

Q3. “gRPC bidi stream between mobile and backend — good idea?” *Strong answer:* mostly no: mobile networks + OS lifecycle kill long streams; gRPC-Web/native gRPC support is awkward; prefer unary gRPC/REST + platform push for wake-ups + SSE/WS for foreground sessions; reserve gRPC bidi for server-to-server (xDS-style) where deadlines/contracts/stable networks apply.